EXAMENS_DEVOPS

Thème

Bibliothèque Numérique du Dakar Institut Technologie

GROUPE 5

Les Membres

1. Moussa CAMARA
2. Eudoxie ABOUTA
3. Azizatou BALDE
4. Frederic CHACKVROUNE

Introduction

Ce mémoire présente le projet de développement d'une plateforme numérique de gestion de la bibliothèque académique du Dakar Institute of Technology. Ce projet est réalisé par le Groupe 5 dans le cadre de l'examen final du module DevOps. L'objectif est de mettre en œuvre une application multiservices reposant sur une architecture microservices et intégrant les principes DevOps.

1. Contexte et Objectifs

La bibliothèque académique constitue un pilier essentiel pour l'apprentissage et la recherche. Le projet vise à moderniser sa gestion par une solution logicielle qui permet :

- Concevoir une architecture Microservices robuste et scalable ;
- Développer des API REST pour la gestion des livres, utilisateurs et emprunts ;
- Créer une interface utilisateur intuitive et responsive ;
- Conteneuriser l'application avec Docker ;
- Orchestrer les services avec Docker Compose ;
- Mettre en place un pipeline CI/CD avec Jenkins ;
- Automatiser le déploiement de l'application ;
- La gestion des utilisateurs (création de comptes, authentification, profils).
- La gestion du catalogue de livres (ajout, modification, suppression, recherche).
- La gestion des emprunts et retours.
-La mise en place d'un tableau de suivi des réservations.

1-2 Equipe et organisation

Le projet a été réalisé par une équipe de 4 développeurs répartis comme suit :

- 2 développeurs backend (Spring Boot)
- 1 développeur frontend (React)
- 1 DevOps Engineer (Docker, Jenkins, CI/CD

2- ARCHITECTURE DU SYSTEME

L'application est composée de quatre services principaux :

- Users Service (Django) : gestion des comptes et authentification.
- Books Service (Django) : gestion du catalogue de livres.
- Borrowings Service (Django) : gestion des emprunts et retours.
- Frontend (React + Vite + Tailwind**)** : interface utilisateur.

Chaque service possède sa propre base de données SQLite et est orchestré par Docker Compose. La communication se fait via API REST documentée avec Swagger.

2-1 Vue d'ensemble de l'architecture : architecture Microservices

```
┌──────────────
```

```
      ▼              ▼              ▼

┌──────────────────────┐ ┌──────────────────────┐
┌──────────────────────┐
│  PostgreSQL  │ │  PostgreSQL  │ │  PostgreSQL  │
│   livres_db  │ │ utilisateurs_db │ │ emprunts_db  │
└──────────────────────┘ └──────────────────────┘
└──────────────────────┘
```

2-2 Technologies Utilisées

Composant	Technologie	Version	Justification
Backend	Sprint Boot	3.1.5	Framework Java robuste et mature
Frontend	React.js	18.2.0	Bibliothèque JavaScript performante
Base de données	PostgreSQL	15	SGBD relationnel fiable
Conteneurisation	Docker	24.0	Isolation et portabilité
Orchestration	Docker compose	2.23	Gestion multi-conteneurs
CI/CD	Jenkins	2.440	Automatisation du pipeline
API documentation	Swagger/Open API	2.2.0	Documentation interactive

2-3 Communication Interservices

La communication entre les Microservices s'effectue via des API REST synchrones.

Pour garantir la résilience du système, plusieurs mécanismes ont été mis en place :

- Timeouts pour éviter les blocages ;
- Retry mécanismes avec back off exponentiel ;
- Circuit breaker pour les appels critiques ;
- Health checks pour le monitoring;

3. MICROSERVICES

3-1 Livre Services

Le service de gestion des livres est responsable de l'ensemble des opérations liées au catalogue de la bibliothèque.

Fonctionnalités :

- CRUD complet des livres ;
- Recherche multicritères (titre, auteur, ISBN) ;

- Gestion des exemplaires et de leur disponibilité ;
- Catégorisation par genre, langue, éditeur ;
- Génération de statistiques ;

3-2 Utilisateur Services

Ce service gère l'ensemble des utilisateurs de la bibliothèque avec trois types distincts : Étudiant, Professeur et Personnel Administratif.

Fonctionnalités :

- Création et gestion des comptes utilisateurs ;
- Authentification et autorisation ;
- Gestion des profils et des droits ;
- Types d'utilisateurs différenciés ;
- Consultation des historiques d'emprunts ;

3-3 Emprunt Service

Le service d'emprunt orchestre l'ensemble des opérations de prêt et de retour.

Fonctionnalités :

- Enregistrement des emprunts ;
- Gestion des retours ;
- Calcul automatique des pénalités de retard ;
- Détection des retards ;
- Historique complet des emprunts ;
- Notification des échéances ;

4 – API REST

4-1 Endpoint du livre service

Méthode	Endpoint	Description
GET	/api/livres	Liste tous les livres
GET	Api/livres/{id}	Détail d\'un livre
GET	/api/livres/recherche=	Recherche multi-critères
GET	/api/livres/disponibles	Livres disponibles
GET	/api/livres/categorie/{cat} /api/livres/auteur/{auteur}	Livres par catégorie Livres par auteur

POST	/api/livres	Ajouter un livre
PUT	/api/livres/{id}	Modifier un livre
DELETE	/api/livres/{id}	Supprimer un livre
PATCH	/api/livres/{id}/disponibilité	Mettre à jour disponibilité
GET	/api/livres/statistiques	Statistiques des livres

4-2 Endpoint de l'utilisateur service

Méthode	Endpoint	Description
GET	/api/utilisateurs	Liste tous les utilisateurs
GET	/api/utilisateurs/{id}	Profil utilisateur
GET	/api/utilisateurs/matricule/{mat}	Recherche par matricule
GET	/api/utilisateurs/type/{type}	Filtrer par type
GET	/api/utilisateurs/recherche=	Recherche utilisateurs
POST	/api/utilisateurs	Créer un utilisateur
PUT	/api/utilisateurs/{id}	Modifier un utilisateur
DELETE	/api/utilisateurs/{id}	Supprimer un utilisateur
GET	/api/utilisateurs/statistiques	Statistiques utilisateurs

4-3 Endpoint Emprunt service

Méthode	Endpoint	Description
GET	/api/emprunts	Liste tous les emprunts
GET	/api/emprunts/{id}	Détail d'un emprunt
GET	/api/emprunts/utilisateur/{id}	Emprunts d'un utilisateur
GET	/api/emprunts/livre/{id}	Emprunts d'un livre
GET	/api/emprunts/encours	Emprunts en cours
GET	/api/emprunts/retards	Emprunts en retard

POST	/api/emprunts	Nouvel emprunt
PUT	/api/emprunts/{id}/retour	Retour de livre
GET	/api/emprunts/statistiques	Statistiques des emprunt

5- CONTENEURISATION AVEC DOCKER

5-1 Docker files

Chaque Microservices possède son propre Docker file multi-stage :

```
# Docker file pour Livre Service
FROM maven:3.8-openjdk-17 AS build
WORKDIR /app
COPY pom.xml.
RUN mvn dependency: go-offline
COPY src src
RUN mvn package -DskipTests
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Le Dockerfile multi-stage permet de :

- Réduire la taille de l'image finale (sans les outils de build) ;
- Améliorer la sécurité (moins de vulnérabilités) ;
- Optimiser les temps de build.

5-2 Docker compose

L'orchestration des services est gérée par Docker Compose :

```
Version : '3.8'
Services :
 db-livre:
  image : postgres :15-alpine
```

```
    environment:
      POSTGRES_DB: livres_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - postgres_livre_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
  livre-service:
    build: ./backend/livre-service
    ports:
      - "8081:8081"
    depends_on:
      db-livre:
        condition: service_healthy
    environment:
      SPRING_PROFILES_ACTIVE: docker
      DB_HOST: db-livre
  # Services similaires pour utilisateur et emprunt...
  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - livre-service
      - utilisateur-service
      - emprunt-service
volumes:
  postgres_livre_data:
```

postgres_utilisateur_data:

postgres_emprunt_data:

6- PIPELINE CI/CD AVEC JENKINS

6-1 Architecture du Pipeline Jenkins

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                PIPELINE CI/CD                                                 │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                                              │
│  [Checkup] → [Build] → [Tests] → [Docker Build] → [Push] → [Deploy]                                          │
│     │         │         │         │           │         │         │                                          │
│     ▼         ▼         ▼         ▼           ▼         ▼         ▼                                          │
│  GitHub    Maven    Unit Tests   Images    Registry    Docker                                                 │
│                    Intégration            Docker Hub   Compose                                               │
│                                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

6-2 Etape du pipeline

Etapes	Actions	Durée Estimée
Checkup	Récupération du code depuis GitHub	30s
Build Backend	Compilation Maven des 3 services	3 min
Build Frontend	npm install && npm run build	2 min
Tests	Tests unitaires et d\'intégration	2 min
Docker Build	Construction des 4 images Docker	2 min
Push	Publication sur Docker Hub	1min

Deploy	Docker-compose up -d	30s
Health Check	Vérification des endpoints	30s

6-3 Jenkins file

```
pipeline {
    agent any

    environment {
        DOCKER_REGISTRY = 'docker.io/bibliotheque'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/dit/bibliotheque-numerique.git'
            }
        }

        stage('Build Backend') {
            parallel {
                stage('Livre Service') {
                    steps {
                        dir('backend/livre-service') {
                            sh './mvnw clean package'
                        }
                    }
```

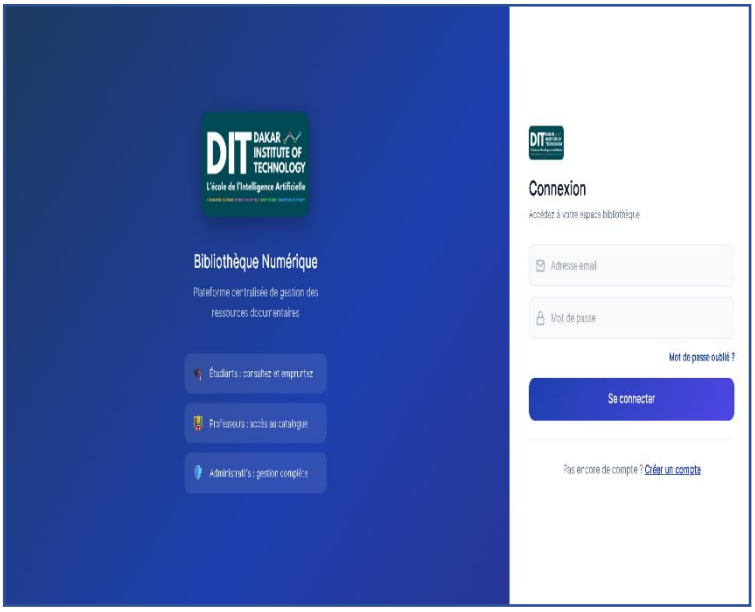
```
        }
        stage('Utilisateur Service') { ... }
        stage('Emprunt Service') { ... }
    }
}

stage('Build Frontend') {
    steps {
        dir('frontend') {
            sh 'npm install && npm run build'
        }
    }
}

stage('Tests') {
    parallel {
        stage('Backend Tests') { ... }
        stage('Frontend Tests') { ... }
    }
}

stage('Docker Build & Push') { ... }

stage('Deploy') {
    steps {
        sh 'docker-compose up -d'
    }
```

```
        }

        stage('Health Check') {
            steps {
                sh '''
                    curl -f http://localhost:8081/api/livres
                    curl -f http://localhost:8082/api/utilisateurs
                    curl -f http://localhost:8083/api/emprunts
                '''
            }
        }
    }

    post {
        success {
            slackSend(color: 'good', message: ' Déploiement réussi!')
        }
        failure {
            slackSend(color: 'danger', message: ' Pipeline échoué!')
        }
    }
}
```

7. INTERFACE UTILISATEUR

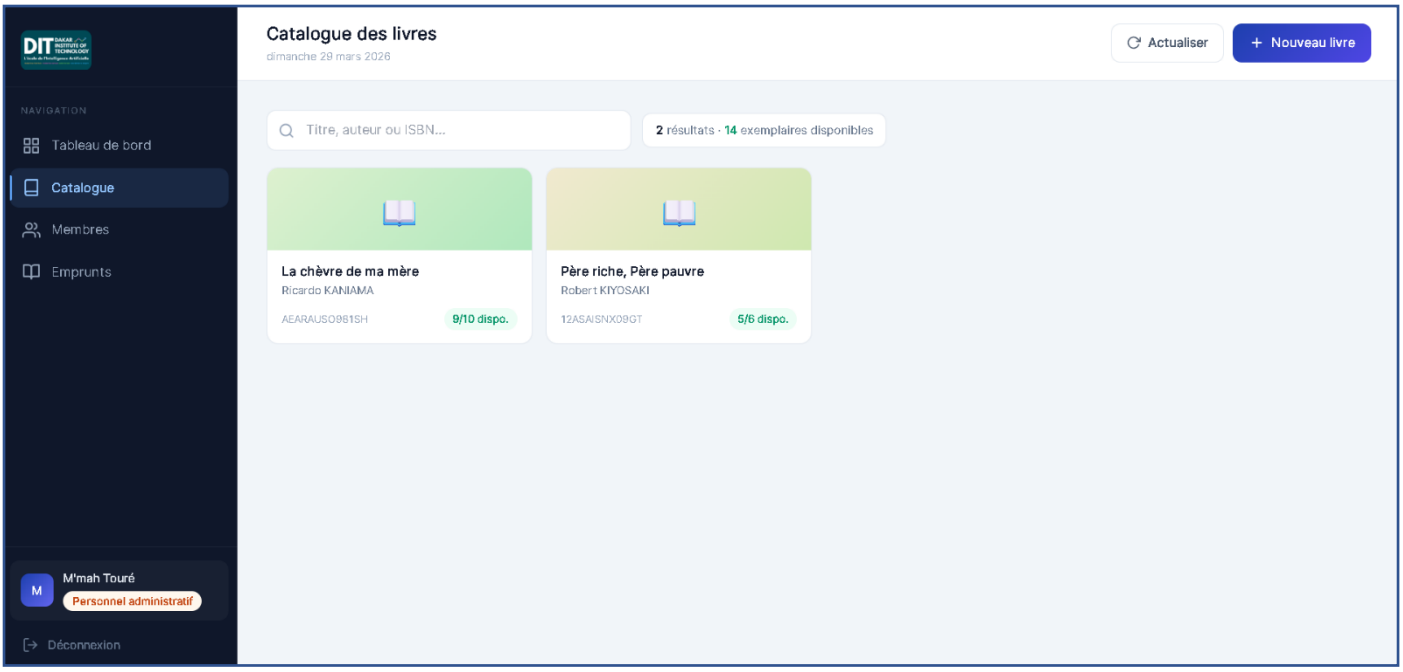
7.1 Création de compte utilisateur

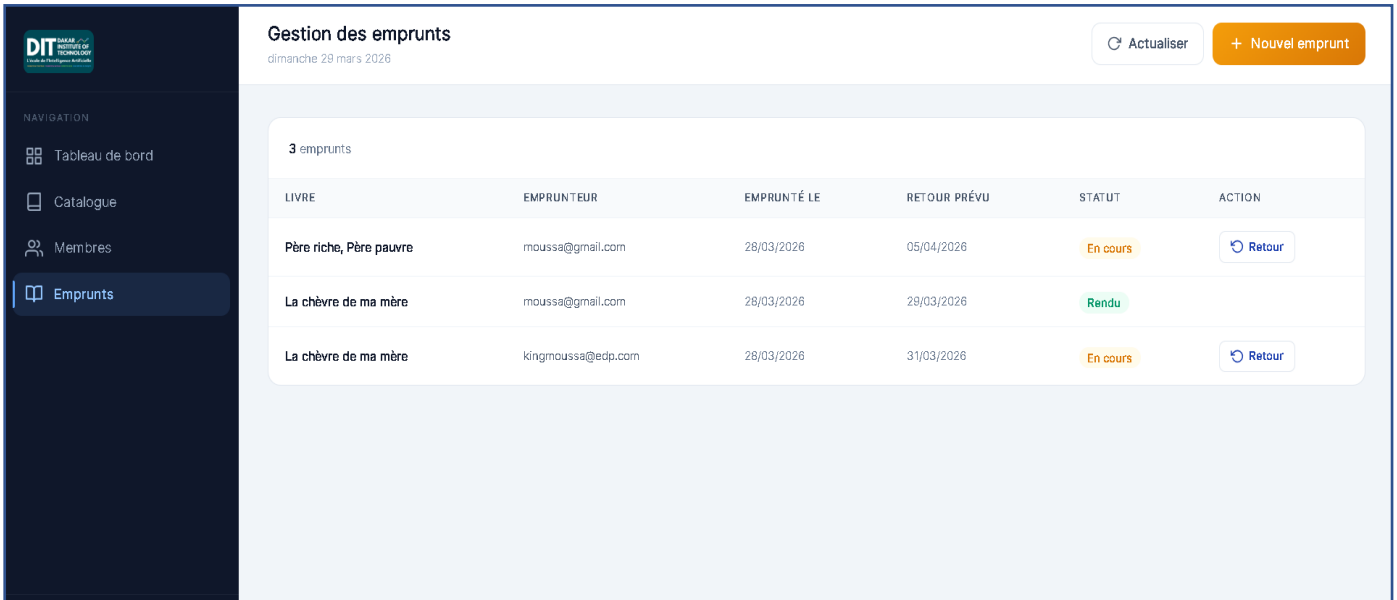
- Les utilisateurs peuvent créer un compte en fournissant leurs informations personnelles.
- Trois rôles sont disponibles : Étudiant, Professeur, Administratif.
- Authentification par email et mot de passe.



7.2 Catalogue de livres

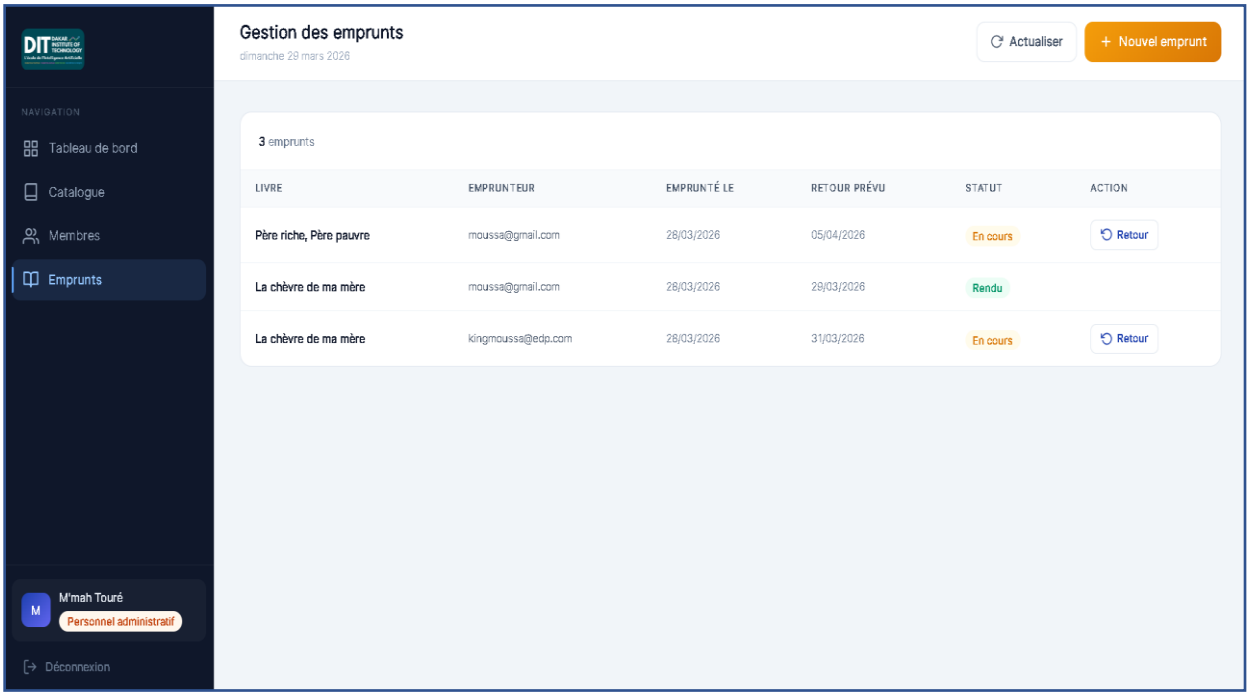
- Consultation du catalogue complet.
- Recherche par titre, auteur ou ISBN.
- Ajout, modification et suppression de livres (réservé aux administratifs).
- Filtrage des ouvrages disponibles.





7.4 Tableau des réservations

- Visualisation des emprunts en cours et des réservations.
- Tableau listant les utilisateurs, les livres empruntés et les dates de retour prévues.
- Permet aux administratifs de suivre l'activité.



Couche	Technologies principales
Backend	Python 3.10, Django 5, Django REST Framework
Frontend	React 18, Vite, Tailwind CSS
Base de données	SQLite (par service)
Conteneurisation	Docker, Docker Compose
CI/CD	Jenkins
Documentation API	Swagger
Proxy	Nginx

9. Pipeline CI/CD

Le pipeline Jenkins est défini en 5 étapes :

1. Récupération du code depuis Git.
2. Exécution des tests sur les trois microservices.
3. Build des images Docker.
4. Déploiement avec Docker Compose.
5. Vérification via health-check.

10. Tests et Qualité

- Tests unitaires et d'intégration pour chaque microservice.
- Utilisation de mocks pour isoler les appels inter-services.
- Vérification de la robustesse et de la cohérence des données.

11. Compatibilité et Déploiement

- Compatible avec Windows, macOS et Linux.
- Recommandation : privilégier Docker Compose pour un déploiement homogène.
- Ajustements spécifiques pour Windows (WSL2, gestion des ports) et Apple Silicon (images ARM/amd64).

12. Discussion et Perspectives

Ce projet illustre la mise en œuvre des principes DevOps : modularité, automatisation, CI/CD. Les perspectives incluent :

- Migration vers une base de données plus robuste (PostgreSQL).
- Ajout de fonctionnalités avancées (notifications, statistiques d'utilisation).
- Déploiement sur le cloud pour une meilleure scalabilité.

13. Conclusion

Le projet de Bibliothèque Numérique du DIT démontre la capacité des étudiants à appliquer les concepts DevOps dans un contexte réel. L'architecture microservices, l'intégration continue et le déploiement automatisé constituent des acquis majeurs. Ce travail ouvre la voie à une gestion moderne et efficace des ressources académique.

13-1 Bilan du Projet

Le projet de bibliothèque numérique Microservices a été réalisé avec succès, atteignant l'ensemble des objectifs fixés. Les principaux résultats sont :

- 3 Microservices fonctionnels et indépendants
- API REST complètes et documentées (Swagger)
- Interface utilisateur moderne et responsive
- Conteneurisation complète avec Docker
- Orchestration avec Docker Compose
- Pipeline CI/CD automatisé avec Jenkins
- Tests unitaires et d'intégration
- Documentation technique et utilisateur

13-2 Compétence Acquises

Ce projet a permis de développer et consolider les compétences suivantes :

- Conception d'architecture Microservices
- Développement d'API REST avec Spring Boot
- Création d'interfaces React.js
- Conteneurisation et orchestration Docker
- Automatisation CI/CD avec Jenkins
- Gestion de projet avec Git/GitHub
- Collaboration en équipe

13-3 Perspective d'amélioration

Plusieurs axes d'amélioration sont envisageables :

- Implementation d'un API Gateway (Spring Cloud Gateway)
- Ajout d'un circuit breaker (Resilience4j)
- Mise en place d'un cache distribué (Redis)
- Monitoring avec Prometheus/Grafana
- Centralisation des logs (ELK Stack)
- Déploiement sur Kubernetes
- Authentification JWT centralisée.